

Wamu Uganda — Revised Project Plan

Document Type: Strategic & Execution Plan

Version: 2.0

Status: Revised for Transaction-First Build

Prepared by: Senior Technical Lead

Date: 2026-09-27

1. Executive Summary

Wamu is Uganda's everyday app: one account for communication, local business discovery, commerce, delivery, and mobility. The original vision was correct, but the execution sequence was wrong. Building messaging first would compete directly with WhatsApp without offering a compelling reason to switch.

This revised plan flips the strategy. Wamu will build the **transaction loop first**:

Discover → Chat → Buy → Pay → Deliver → Review

The first production MVP will prove one complete, repeatable value loop: a customer orders from a merchant, pays by mobile money or cash, and a verified rider delivers it. Messaging, short video, and passenger mobility follow only after this loop is stable and repeatable.

The goal is not to clone WhatsApp, TikTok, or SafeBoda. The goal is to make them obsolete inside one connected ecosystem by making every interaction transactional.

2. Strategic Reframe

2.1 What Wamu Is Not

- Not a general-purpose messenger.
- Not an entertainment-first short-video platform.
- Not a standalone ride-hailing app.
- Not a collection of disconnected business tools.

2.2 What Wamu Is

Wamu is Uganda's transaction layer.

It connects local businesses, customers, payments, and verified riders into one seamless experience. Chat, video, and rides are features that support transactions — not the other way around.

2.3 Why Users Will Switch

Competitor	What They Do Well	How Wamu Wins
WhatsApp	Messaging, groups, calls	Business chat tied to orders, payments, and delivery tracking.
TikTok	Short video, entertainment	Shoppable business videos with direct “Buy” buttons.
SafeBoda	Rides, delivery	One rider network for delivery and rides; higher rider earnings.
Business apps	Catalogs, orders	Unified discovery, chat, payment, and delivery in one app.

Users will not leave WhatsApp because Wamu has chat. They will leave because Wamu makes business chat **more useful** — it is connected to orders, payments, and delivery.

3. Product Vision

One Wamu account. Multiple verified capabilities.

- Customer
- Business Owner
- Verified Rider

A person registers once. Additional capabilities are enabled and verified on the same account.

3.1 Core Principles

- **Transaction-first:** Every feature must support a real transaction.
- **Uganda-first:** Designed for local payments, connectivity, and mobility.
- **Trust by design:** Verification, security, and privacy built into the architecture.

- **Shared infrastructure:** One rider network, one payment abstraction, one location service.
- **Phased expansion:** Prove one loop, then add reasons to stay.

4. The Core Value Loop

Discover → Chat → Buy → Pay → Deliver → Review

This is the heart of Wamu. Everything else is an extension.

1. **Discover:** Customer finds a local business or product.
2. **Chat:** Customer talks to the business or places an order directly.
3. **Buy:** Customer selects products or services.
4. **Pay:** Customer pays via MTN MoMo, Airtel Money, or cash.
5. **Deliver:** A verified rider picks up and delivers the order.
6. **Review:** Customer rates the business and rider.

Once this loop works repeatedly, Wamu adds:

- Transactional chat
- Short business video discovery
- Passenger mobility
- Merchant settlement and payouts
- Intelligence and personalization

5. Beachhead Strategy

Start in Kampala. Start with food and groceries.

Why food and groceries?

- High frequency
- Clear delivery need
- Existing boda delivery behavior
- Easy to measure repeat rate
- Simpler catalog than general retail

Expand to:

1. Pharmacy
2. Retail and electronics
3. General commerce
4. Passenger mobility

Do not launch nationwide. Own one city first.

6. Revised MVP Definition

The first production MVP must prove the transaction loop. It must not attempt the entire long-term platform.

6.1 In Scope

Customer App

- Phone OTP login
- Browse merchants by category and nearby
- View products and services
- Chat with merchant or place order
- Pay by MTN MoMo, Airtel Money, or cash on delivery
- Track delivery
- Rate order and rider

Merchant App / Web

- Onboarding and verification
- Store profile
- Product catalog
- Receive, accept, or reject orders
- Chat with customer
- Basic sales analytics

Rider App

- Onboarding
- ID, selfie, vehicle, and plate verification

- Availability toggle
- Accept delivery
- Navigate to pickup and dropoff
- Proof of delivery
- Earnings summary

Admin Panel

- Verify merchants and riders
- Manage disputes and refunds
- Monitor fraud
- Basic content moderation

6.2 Out of Scope for MVP

- General 1:1 messaging
- Groups and communities
- Status/stories
- Voice and video calls
- Short video Discover feed
- Passenger transport
- ML-based recommendations
- Automated merchant settlement and rider payouts

These are future phases.

7. Phased Roadmap

Phase	Timeline	Scope	Goal
Phase 0: Concierge	Weeks 0–6	10 merchants, 20 riders, manual dispatch.	Validate delivery demand.
Phase 1: Commerce & Delivery MVP	Months 2–5	Customer, merchant, rider apps. Orders, MoMo/cash, tracking, ratings.	100 deliveries/day in Kampala.
Phase 2: Transactional Chat	Months 5–8	In-order chat, business chat, groups for merchants.	Businesses use Wamu as primary customer channel.
Phase 3: Discover Short Video	Months 8–11	Business videos, vertical feed, likes, comments, follows, direct buy.	Users come for business discovery.
Phase 4: Passenger Mobility	Year 2	Ride requests, matching, navigation, safety.	Riders earn more; customers use one app.
Phase 5: Full Super App	Year 2+	Payments, payouts, settlement, ML, fraud detection.	Everything on Wamu.

8. Functional Specifications

8.1 Account & Capability Model

A person registers once. Additional capabilities are enabled and verified on the same account.

- Wamu Account
- User
- Customer
- Business Owner (optional)
- Verified Rider (optional)

8.2 Order Lifecycle

PENDING → CONFIRMED → PROCESSING → READY → OUT_FOR_DELIVERY → DELIVERED

Cancellation:

PENDING → CANCELLED
 CONFIRMED → CANCELLED

Refund:

DELIVERED → REFUNDED

Order state transitions must be controlled by the backend. The mobile client must never be trusted to mark an order delivered or a payment successful.

8.3 Delivery States

REQUESTED → RIDER_ASSIGNED → ACCEPTED → GOING_TO_PICKUP → PRODUCT_COLLECTED → IN_TRANSIT



8.4 Rider Verification States

DRAFT → SUBMITTED → UNDER_REVIEW → APPROVED / REJECTED → RESUBMIT → SUSPENDED / EXPIRED



8.5 Rider Operating States

OFFLINE → AVAILABLE → REQUEST_RECEIVED → ACCEPTED → EN_ROUTE_PICKUP → AT_PICKUP → IN_TRANSIT



9. Technical Architecture

9.1 Stack

- **Mobile:** Flutter (Android-first)
- **Backend:** Python + FastAPI
- **Database:** PostgreSQL
- **ORM:** SQLAlchemy 2.x + Alembic
- **Validation:** Pydantic v2
- **Cache/Queue:** Redis

- **Real-time:** WebSockets
- **Media:** S3-compatible object storage + CDN
- **Push:** Firebase Cloud Messaging
- **Maps:** Google Maps behind Location Service abstraction
- **Payments:** MTN MoMo, Airtel Money, cards behind Payment Provider abstraction
- **Deployment:** Docker, Nginx, CI/CD
- **Monitoring:** Sentry, Prometheus/Grafana
- **No Kubernetes initially**

9.2 Architecture Pattern

Modular monolith, not microservices.

Bounded contexts:

- Identity
- Catalog
- Orders
- Payments
- Delivery
- Mobility (future)
- Discovery (future)
- Messaging (future)
- Notifications
- Admin/Audit

9.3 Key Patterns

- Outbox pattern for reliable events
- Idempotency keys for payments and orders
- State machines for order, delivery, ride
- Webhook signature verification
- Feature flags for staged rollout
- Audit logs for sensitive access

- Soft deletes + retention rules

9.4 Initial API Structure

```
POST /api/v1/auth/request-otp
POST /api/v1/auth/verify-otp
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
GET /api/v1/auth/me
GET /api/v1/businesses
GET /api/v1/products
POST /api/v1/orders
GET /api/v1/orders/{id}

POST /api/v1/deliveries
GET /api/v1/deliveries/{id}
POST /api/v1/deliveries/{id}/accept
POST /api/v1/deliveries/{id}/pickup
POST /api/v1/deliveries/{id}/complete

POST /api/v1/riders/application
POST /api/v1/riders/documents
GET /api/v1/riders/me
PATCH /api/v1/riders/availability
```

10. Security, Privacy & Compliance

10.1 Security Controls

- Never store raw OTPs.
- Hash refresh tokens.
- Never log OTPs or tokens.
- Private encrypted storage for identity documents.
- Strict authorization for verification data.
- Audit logs for sensitive access.
- Rate limiting by phone, IP, and device.
- File size/type/content validation.
- Server-side authorization.

- Backup and restore testing.
- Fake-account and duplicate-identity controls.
- GPS manipulation detection.
- Fake-trip and payment-fraud controls.
- Delivery dispute workflows.
- Fake-review monitoring.
- Suspicious-device-change monitoring.

10.2 Privacy

- Detailed location history should not be retained forever.
- Active-trip location should be available only for the relevant operational and safety period.
- Government ID images and sensitive verification records must never be publicly displayed.
- Customers see a safe verified profile: name, photo, verified badge, vehicle type, plate, rating, trip count, Wamu rider reference.

10.3 Compliance

- Uganda Data Protection and Privacy Act 2019
- Consent, data subject rights, retention
- KYC/AML for payments
- Transport licensing for passenger mobility
- Rider insurance and liability
- Content moderation for Discover
- Payment provider commercial agreements

11. Payments

11.1 Payment Flow

Customer → Create Order → Create Payment → Provider
Customer Confirmation → Provider Webhook → Wamu Verification → Payment SUCCESS → Or

11.2 Requirements

- Payment-provider abstraction
- Server-side payment confirmation
- Webhook signature verification
- Idempotency
- Duplicate webhook protection
- Refund handling
- Transaction reconciliation

11.3 Supported Providers

- MTN Mobile Money
- Airtel Money
- Cards (subject to API/commercial availability)

12. Delivery & Rider Network

12.1 Delivery Flow

Customer → Business → Delivery Request → Rider → Customer

Steps:

- Customer delivery address
- Business pickup location
- Delivery request
- Distance and ETA
- Delivery price estimate
- Rider matching
- Rider acceptance
- Pickup confirmation
- Live tracking
- Delivery completion

- Proof of delivery
- Ratings

12.2 Rider Matching

Initial matching should be simple and explainable:

- Verified status
- Availability
- Service capability
- Distance to pickup
- ETA
- Vehicle type
- Current trip status
- Service area

Then become more data-driven later.

12.3 Pricing

Backend Pricing Engine:

```
Delivery = base + distance + time + service factors
Passenger = base + distance + time + vehicle factors
```

Flutter requests an estimate but does not define the final fare.

13. Future Modules

13.1 Transactional Chat

- In-order chat
- Business chat
- Groups for merchants
- Payments, receipts, delivery updates inside chat
- Goal: replace WhatsApp for business communication

13.2 Discover Short Video

- Business videos
- Vertical feed
- Likes, comments, shares, saves, follows
- Search and category discovery
- Local/nearby discovery
- Direct buy button
- Goal: replace TikTok for business discovery

13.3 Passenger Mobility

- Pickup and destination
- Distance and ETA
- Estimated fare
- Rider matching and acceptance
- Live tracking
- Navigation
- Trip history
- Payment
- Passenger/rider ratings
- Safety: Share Trip, Call/Chat Rider, Report Problem, Emergency Help
- Goal: replace SafeBoda for rides

14. Metrics

North Star: Completed deliveries per day.

Supporting metrics:

- Order completion rate
- Average delivery ETA
- Repeat customer rate
- Merchant retention
- Rider utilization

- Payment success rate
- CAC per customer
- Take rate per order
- Safety incidents per 1,000 trips
- Daily/monthly active users
- Messages sent and delivered
- Video uploads and qualified discovery
- Product clicks from Discover
- Completed rides

15. Risks & Mitigation

Risk	Impact	Mitigation
Too many features at once	High	Release in phases around the core loop.
Poor connectivity	High	Low-data mode, compression, resumable uploads.
Fake identities/fraud	High	OTP, verification, device controls, risk rules.
Sensitive data exposure	High	Private storage, encryption, authorization, audit logs.
Weak recommendations	Medium	Start rule-based; add ML after collecting data.
Payment disputes	High	Server confirmation, webhooks, idempotency, reconciliation.
Rider safety	High	Verification, tracking, reporting, safety controls.
Operational complexity	High	Modular architecture, simple initial infrastructure.
Regulatory issues	High	Legal review for payments, transport, data protection.

16. Operational Plan

16.1 Concierge Phase

- 10 merchants
- 20 riders
- Manual dispatch via WhatsApp/phone

- Prove delivery works
- Measure repeat orders
- No app yet

16.2 Onboarding

- Merchant onboarding and photography
- Rider onboarding and training
- Customer support
- Dispute resolution
- Refunds and reconciliation
- Fraud monitoring
- Safety incident response
- Rider earnings and payouts

16.3 Support

- In-app chat support
- Phone support for merchants and riders
- Dispute workflow
- Refund workflow
- Safety escalation

17. Go-to-Market

17.1 City Launch

- Kampala first
- Food and groceries first
- Then pharmacy, retail, general commerce

17.2 Merchant Acquisition

- Direct sales
- Referral incentives

- Free onboarding and photography
- Competitive take rate

17.3 Rider Recruitment

- Referral bonuses
- Higher earnings through delivery + rides
- Fast verification
- Weekly payouts

17.4 Customer Acquisition

- Referral codes
- First-order discounts
- Social media
- Local influencers
- Business partnerships

18. Conclusion

Wamu's ideal is correct: one app for chat, discovery, commerce, delivery, and mobility. But the execution must be transaction-first.

Build the loop:

Discover → Chat → Buy → Pay → Deliver → Review

Then add chat.

Then add video.

Then add rides.

Then add everything.

That is how Wamu gets users off WhatsApp, TikTok, and SafeBoda.

Not by cloning them.

By making them obsolete inside one connected ecosystem.

Build the loop, not the features.

Appendix A — Database Tables (MVP)

Core: users, user_profiles, devices, sessions, otp_challenges, businesses, business_profiles, categories, products, services, orders, order_items, payments, payment_events.

Delivery: rider_profiles, rider_verifications, rider_documents, rider_vehicles, rider_vehicle_documents, rider_service_types, rider_service_areas, rider_availability, rider_status_history, delivery_requests, delivery_events.

Mobility (future): ride_requests, ride_trips, trip_locations, rider_ratings.

Messaging (future): conversations, conversation_members, messages, message_reactions, message_attachments, groups, communities, statuses.

Discover (future): business_videos, video_assets, video_categories, video_hashtags, video_views, video_watch_events, video_likes, video_comments, video_shares, video_saves, business_followers, user_interest_profiles, recommendation_candidates, recommendation_impressions, user_feedback, video_reports.

Appendix B — Environment & Deployment

- Docker
- Nginx
- CI/CD pipeline
- Staging and production environments
- PostgreSQL managed service
- Redis managed service
- S3-compatible object storage
- CDN
- Sentry
- Prometheus/Grafana
- Firebase Cloud Messaging
- Maps API
- Payment provider APIs

No Kubernetes in first deployment unless real operational scale requires it.

